

Mergen Hareketli Uydu Terminali - Detayli Proje Raporu

Proje Hakkimda

Bu proje, UniS yazılım ekibi tarafından geliştirilen hareketli bir uydu terminali sistemidir. Terminalin amacı, hareketli bir platform üzerinde (arac, gemi vb.) antenin boresight eksenini hedef uyduya (Turksat 4B / Turksat 5A) kilitli tutmaktır. Sistem; IMU, GPS, encoder, TLE/SGP4 ve lazer/QPD geri beslemesini birlikte kullanan kapalı dongu bir kontrol mimarisine sahiptir.

Takim Uyeleri ve Katkileri

Uye	Klasor	Katkı Alanı
Kerim	`kerim unis/BTK_Satellite_Terminal/`	Gömülü C++ yazılımı: SGP4, EKF, PID, baslatma, telemetri
Efe	`Efe/`	ROS 2 arayuzu, dataset simulator, firmware referansı, mimari
Elif (Simge)	`simge_elif/`	C++ stabilizasyon ornegi, low-pass filtre, PID

Klasor Yapisi

```
unis/
├── README.md # Proje genel bilgilendirme
├── PID_Algoritma.py # Bos dosya
├── (placeholder)
├── run_windows_dataset.bat # Dataset simulator calistirma
├── run_windows_gui.bat # GUI calistirma
├── run_windows_report_pdf.bat # PDF rapor olusturma
├── kerim unis/
├── genel bilgi.md # Sistem akisi dokumani
├── BTK_Satellite_Terminal/ # Gömülü C++ kodu
├── CMakeLists.txt
├── README.md
├── include/ # Baslik dosyalari
├── types.h # Veri yapilari ve enum'lar
├── config.h # Sistem sabitleri
├── initialization.h # Baslatma, homing, kalibrasyon
├── SensorFusion.h # EKF (Extended Kalman Filter)
├── SatelliteTracker.h # SGP4 propagator + koordinat donusum
├── PIDController.h # PID kontrol + guvenlik
├── telemetry.h # Telemetri yayini + komut alici
├── src/
├── main.cpp # Ana kontrol dongusu
├── SensorFusion.cpp # (bos)
├── SatelliteTracker.cpp # (bos)
├── PIDController.cpp # (bos)
├── scripts/ # (bos)
├── rtos/ # (bos)
├── docs/ # (bos)
├── Efe/
├── README.md # Mimari aciklamasi
├── tools/
├── dataset_simulator.py # CAD oncesi dogrulama araci
├── reports/
├── OTR_Yazilim_Raporu.pdf # On tasarim raporu PDF
├── OTR_Yazilim_Raporu.md # On tasarim raporu kaynagi
├── generate_pdf.py # PDF olusturma scripti
├── datasets/
├── sample_motion.csv # Ornek hareket verisi
├── sample_result.csv # Ornek sonuc verisi
├── mergen_ws/ # ROS 2 workspace
├── src/mergen_interfaces/ # ROS 2 servis tanimlari
├── CMakeLists.txt
├── package.xml
├── srv/
├── SetMode.srv # Mod degistirme servisi
├── SaveParameters.srv # Parametre kaydetme servisi
├── firmware_reference/
├── teensy_stm32_cpp/ # Teensy/STM32 referans kodu
├── README.md
├── include/
├── pid_controller.hpp
├── kalman_filter.hpp
├── src/
├── main_loop_example.cpp
├── docs/
├── sistem_mimarisi.md # 4 katmanli mimari dokumani
├── sartname_karsilama_matrisi.md # Sartname gereksinimleri
├── gazebo_yol_haritasi.md # CAD/Gazebo entegrasyon plani
├── arayuz/ # Python Tkinter GUI
├── main.py # Ana GUI uygulaması
├── requirements.txt # Bagimliliklar
├── README.md # Arayuz dokumani
├── core/
├── satellite_pointing.py # Uydu yonelim hesaplama
├── ros_bridge/
├── client.py # ROS 2 istemci (simulasyon stub)
├── simge_elif/
├── x.cpp # C++ stabilizasyon ornegi
```

Sistem Mimarisi

Sistem 4 ana katmandan oluşur:

```
├── KATMAN 4: Arayuz (GUI + ROS 2)
├── Manuel/Otomatik mod, GPS girisi, uydu secimi, telemetri izleme, parametre kaydi
├── KATMAN 3: Kontrol (PID + Guvenlik)
├── Azimuth PID | Elevasyon PID | Stabilizasyon PID
├── Lazer takip PID | Spiral arama | Limit kontrolu
├── KATMAN 2: Kestirim (EKF + SGP4 + Koordinat)
├── IMU fuzyonu | Uydu konumu | Platform eim telafisi
```

```
KATMAN 1: Algilama (IMU +  
GPS + Encoder + QPD) | 100 Hz IMU | 1 Hz GPS | Her cycle encoder | QPD
```

Veri Akisi

IMU/Gyro -> Kalman + Quaternion -> Stabilizasyon PID -> X/Y motor komutu
GPS + Uydu parametresi -> Azimuth/Elevasyon hedefi -> Motor PID -> Az/El motor komutu
QPD/Lazer -> Hata vektörü -> Aktif takip PID -> Az/El offset düzeltmesi
Encoder -> Gerçek motor konumu -> PID geri beslemesi
GUI -> Mod/hedef/parametre -> Kontrol node'leri

1. Gömülü C++ Yazilimi (Kerim)

1.1 Veri Yapilari (types.h)

Temel matematik tipleri:

- `Vec3` - 3 boyutlu vektor (float)
- `Vec3d` - 3 boyutlu vektor (double)
- `Quaternion` - Quaternion (w, x, y, z) ile carpma, normalize, conjugate

Cografi koordinatlar:

- `LLA` - Enlem/Boylam/Irtifa (derece, metre)
- `ECEF` - Earth-Centered Earth-Fixed (metre)
- `AzEl` - Azimut/Elevasyon/Mesafe

Sensor veri yapilari:

- `IMURawData` - ivmeolcer (m/s^2) + jiroskop (rad/s) + timestamp
- `GPSData` - LLA konum + hiz + yon + HDOP + uydu sayisi
- `TLEData` - TLE yorunge verisi (tum alanlar)
- `SGP4Elements` - SGP4 onceden hesaplanmis elemanlar

Islenmis veri:

- `Orientation` - Quaternion + Roll/Pitch/Yaw + gyro bias
- `AntennaTarget` - Hedef aci setpoint + uydu bilgisi

Kontrol:

- `PIDState` - Integral, onceki hata, turev, output
- `MotorCommand` - Eksen + yon + PWM gorev dongusu + hiz
- `EncoderData` - Az/El gercek acilar + hiz + timestamp

Durum makinası:

- `SystemState` - POWER_ON, HW_SELF_TEST, HOMING, CALIBRATING, IDLE, AUTO_TRACKING, MANUAL, ERROR
- `ErrorCode` - IMU_FAULT, GPS_NO_FIX, AZ_MOTOR_FAIL, EL_MOTOR_FAIL, TLE_INVALID, AZ_LIMIT_HIT, EL_LIMIT_HIT, WATCHDOG

Telemetry:

- `TelemetryPacket` - 70 byte'lik UART paketi (magic, timestamp, state, errors, konum, yonelim, acilar, hatalar, RSSI, CRC16)

- `UserCommand` - SET_AUTO_MODE, SET_MANUAL, MANUAL_AZ, MANUAL_EL, LOAD_TLE, HOME, EMERGENCY_STOP
- `CommandPacket` - Komut + payload union

1.2 Sistem Sabitleri (config.h)

Sabit	Deger	Aciklama
MAIN_LOOP_PERIOD_MS	10	100 Hz ana dongu
TELEMETRY_PERIOD_MS	100	10 Hz telemetri
GPS_UPDATE_PERIOD_MS	1000	1 Hz GPS
TLE_UPDATE_PERIOD_S	3600	Saatlik TLE
IMU_SAMPLE_RATE_HZ	100	IMU ornekleme hizi
GYRO_NOISE_SIGMA	0.005	Jiroskop gurultu std
ACCEL_NOISE_SIGMA	0.01	lvmeolcer gurultu std
GRAVITY_MS2	9.80665	Yercekimi sabiti
CALIBRATION_SAMPLES	500	5 sn kalibrasyon
AZ_LIMIT_MIN/MAX	0 / 360	Azimut fiziksel limit
EL_LIMIT_MIN/MAX	0 / 90	Elevasyon fiziksel limit
AZ_MAX_SPEED_DPS	60	Azimut max hiz (derece/s)
EL_MAX_SPEED_DPS	30	Elevasyon max hiz (derece/s)
AZ_KP/AZ_KI/AZ_KD	2.5 / 0.1 / 0.3	Azimut PID katsayilari
EL_KP/EL_KI/EL_KD	3.0 / 0.08 / 0.25	Elevasyon PID katsayilari
PID_DEADBAND_DEG	0.1	Deadband esigi
DERIV_FILTER_ALPHA	0.7	Turev filtre katsayisi
RSSI_LOCK_THRESHOLD_DB	-90	Sinyal kilit esigi
WATCHDOG_TIMEOUT_MS	500	WDT suresi

1.3 Baslatma ve Kalibrasyon (initialization.h)

Donanim Oz-Testi (HardwareSelfTest):

- IMU kontrolu (I2C adresi 0x68, MPU-6050)
- GPS modulu kontrolu (UART, 9600 baud)
- Azimut motor + encoder testi
- Elevasyon motor + encoder testi
- IMU ivmeolcer kalibrasyon dogrulama ($9.8 \pm 2 \text{ m/s}^2$)

Homing (HomingProcedure):

- Her iki eksen icin limit switch'e yaklasma
- CW yonunde limit switch'e git (15 sn timeout)
- ~2 derece geri cekilme (mekanik gerilim azaltma)
- Encoder sifirlama (HOME referans noktasi)

IMU Kalibrasyonu (IMUCalibration):

- 500 ornek (~5 sn) hareketsiz ortalama

- Gyro bias: ortalama gyro degeri
- Accel bias: ortalama - normalize edilmiş yercekimi
- Kalite kontrolu: %5 olcekleme toleransi, <0.1 rad/s bias

Orkestrator (system_initialize):

- Test -> Homing -> Kalibrasyon sirasiyla calistir
- Herhangi bir asamada basarisizlik -> ERROR modu

1.4 Sensör Füzyonu - EKF (SensorFusion.h)

7 boyutlu Extended Kalman Filter:

- Durum: [q0, q1, q2, q3, bx, by, bz] (quaternion + gyro bias)
- Predict: Gyro okumasi ile quaternion kinematigi
- $\dot{q} = 0.5 * \Omega(w_{corrected}) * q$
- $\omega_{corrected} = \omega_{measured} - bias$
- 7x7 Jacobian F ile kovaryans yayilimi
- Update: Ivmeölcer olcumu ile düzeltme
- Beklenen ivme = $R(q) * [0, 0, g]$
- Inovasyon: $accel_{raw} - h(x)$
- 3x7 ölçüm Jacobian H
- Kalman kazanci $K = P * H^T * S^{-1}$
- Durum + kovaryans güncellemesi
- Güçlü ivme detekte: $|accel_{mag} - g| > 3.0$ ise update atlanir
- Quaternion normalize ve kovaryans izi ile yakinlasma kontrolu

Cikti: Quaternion -> Roll/Pitch/Yaw (ZYX sirasi)

1.5 Uydu Takibi - SGP4 (SatelliteTracker.h)

TLE Parser:

- İki satir TLE formatini TLEData'ya donusturur
- Epoch yil (2-digit -> 4-digit), BSTAR decode, eccentricity

SGP4 Propagator:

- Hoots & Roehrich (1980), Vallado (2013) uyarlamasi
- TLE'den ortalama elemanlari turetme (bir kez)
- Newton-Raphson ile Kepler denklemi cozumu (8 iterasyon)
- J2/J3 perturbasyon düzeltmeleri
- Cikti: ECI koordinatlarinda konum (km) + hiz (km/s)

Koordinat Donusumleri:

- ECI -> ECEF (Dunya donusu, GAST)
- ECEF -> LLA (WGS-84, Bowring iterasyonu)
- LLA + ECEF -> Az/EI (NED cercevesi)
- Platform egim telafisi (Roll/Pitch kompanzasyonu)

1.6 PID Kontrol (PIDController.h)

PIDController (tek eksen):

- Anti-windup: Cikis doyuktaysa ve hata ayni yondeyse integral dondurma
- Turev filtreleme: EMA dusuk geciren ($\alpha = 0.7$)
- Deadband: +/- 0.1 derece icinde motor durdurma, integral sifirlama
- Aci sarmalama: Azimut icin -180/+180 derece

SafetyMonitor:

- Hard limit: Fiziksel limit switch -> aninda durdurma
- Soft limit: Hiz %50 kisiltama
- Kablo sarmalama: Kademeli yavaslatma (350-360 derece)
- Acil durum: Switch tetiklenirse emergency_stop

AntennaController (cift eksen yoneticisi):

- Uydu ufkun altindaysa -> park pozisyonu (0, 0)
- Hiz -> PWM donusumu
- Hedefte mi kontrolu (deadband * 2)

1.7 Telemetry (telemetry.h)

TelemetryPublisher (10 Hz):

- ~70 byte'lik UART paketi
- Alanlar: magic, timestamp, state, errors, LLA, roll/pitch/yaw, encoder acilari, hedef acilar, hata acilari, uydu bilgisi, RSSI, CRC16
- CRC-16/CCITT (polinom 0x1021)

CommandReceiver (dairesel kuyruk):

- 8 komutluk FIFO
- UART'tan byte akisi -> CommandPacket
- polling ile her ana dongude okuma

1.8 Ana Kontrol Dongusu (main.cpp)

FreeRTOS veya bare-metal uyumlu super-loop:

```
ADIM 1: Baslatma (bir kez)<br/>- Donanim testi -> Homing -> IMU kalibrasyon<br/>- EKF baslatma<br/>- Varsayilan TLE yukleme (NOAA-19)<br/>-> IDLE durumu -> AUTO_TRACKING'e gecis<br/><br/>ADIM 2: Veri Toplama (her 10ms)<br/>- IMU okuma + bias cikarma (100 Hz)<br/>- GPS okuma + HDOP kontrolu (1 Hz)<br/>- Encoder okuma (her cycle)<br/>- Komut kuyrugu polling (her cycle)<br/><br/>ADIM 3: Sensor Fuzyonu + Koordinat<br/>- EKF predict (gyro) + update (accel)<br/>- SGP4 propagate (TLE -> ECI -> ECEF -> Az/El)<br/>- Platform egim telafisi (roll/pitch kompanzasyonu)<br/>- Anten hedef guncelleme<br/><br/>ADIM 4: Kontrol<br/>- Komut isleme (mode degisikligi, manuel hedef, TLE, homing, emergency)<br/>- Limit switch okuma<br/>- PID + Guvenlik<br/>- Motor PWM uygulama<br/><br/>ADIM 5: Telemetry (10 Hz)<br/>- Paket olusturma ve UART gonderimi<br/>- Takip hatasi denetimi (MAX_TRACKING_ERROR_DEG = 5)
```

2. Dataset ile CAD Öncesi Do■rulama (Efe)

[dataset_simulator.py](#)

CAD modeli hazır olmadan kontrol algoritmasını test etmek için kullanılır.

Ozellikler:

- `--generate`: 5 dakikalık, 100 Hz sentetik veri üretir
- Roll: $8 \cdot \sin(2 \cdot \pi \cdot t / 10) + \text{gauss}(0, 0.18)$
- Pitch: $8 \cdot \cos(2 \cdot \pi \cdot t / 10) + \text{gauss}(0, 0.18)$
- Yaw: $36 \cdot t \bmod 360$
- Hedef: Az=120, El=30 (sabit)
- Sanal QPD: sin/cos ile merkez etrafında salınım
- `--input/--output`: Var olan dataset üzerinde algoritma çalıştırır
- 1D Kalman filtreleme (roll/pitch)
- QPD hata vektörü hesabı
- Azimuth PID + Elevasyon PID
- Stabilizasyon PID (X/Y)
- Hedef kilit durumu tespiti

Cikti: Filtrelenmiş IMU, QPD hata vektörü, PID PWM çıktıları, kilit durumu

3. Python Arayüz (arayüz/)

main.py (Tkinter GUI)

Sartnamedeki bilgisayar/tablet kontrol ihtiyacını karşılar.

Bileşenler:

- Komut ve Mod paneli: Otomatik/Manuel mod seçimi
- Manuel hedef gönderimi (Azimuth + Elevasyon)
- Uydu Yönelimi paneli: GPS enlem/boylam girişi, Türksat 4B/5A seçimi
- Az/El hesaplama butonu (jeostasyoner uydu yaklaşımı)
- Telemeti metin kutusu (durum mesajları)
- Parametre Kaydet (JSON)
- Güvenli Mod butonu

ROS 2 Entegrasyonu:

- Şu an simülasyon modunda (ROS olmayan bilgisayarda çalışır)
- Ubuntu 22.04 + ROS 2 Humble'da rclpy publisher/service client'a genişletilecek

satellite_pointing.py

Jeostasyoner uydu için yaklaşık Az/El hesabı:

- Türksat 4B: 50.0 doğu boylamı
- Türksat 5A: 31.0 doğu boylamı
- Küresel geometri: acos , atan2 ile azimuth/elevasyon

ros_bridge/client.py

Ince ROS 2 istemci:

- set_mode(mode) -> simulasyon mesaji
- send_manual_target(az, el) -> simulasyon mesaji
- save_parameters() -> simulasyon mesaji

4. ROS 2 Servis Tanimlari (Efe/mergen_ws/)

SetMode.srv

```
string mode # "automatic", "manual", "safe"<br/>---<br/>bool accepted<br/>string message
```

SaveParameters.srv

```
string profile_name # Profil adi<br/>---<br/>bool saved<br/>string message
```

5. Firmware Referansi (Efe/firmware_reference/)

Teensy/STM32 mikrodnetleyici icin referans implementasyon:

kalman_filter.hpp

- 1D Kalman Filter
- process_noise (q_) ve measurement_noise (r_) parametreli
- predict + update tek adimda

pid_controller.hpp

- Klasik PID (P + I + D)
- Output clamping
- dt guvenlik kontrolu

main_loop_example.cpp

- 2x Kalman (roll, pitch)
- 4x PID (azimuth, elevation, roll_stab, pitch_stab)
- Statik degiskenlerle hafizada tutma

6. Sartname Karsilama Matrisi

Sartname Istegi	Karsilama	Durum
Azimuth 0-360 derece	Azimuth PID + continuous joint	Tasarlandi
Elevasyon 0-90 derece	Elevation PID + limit kontrolu	Tasarlandi
Manuel mod	Python GUI ManuelCommand + ROS servis	Tasarlandi
Otomatik mod	IMU + Kalman + QPD + PID state machine	Tasarlandi
IMU/Gyro stabilizasyon	KalmanFilter1D + quaternion + stabilization PID	Tasarlandi

Lazer tabanlı takip	QPD hata vektörü + aktif geri besleme PID	Tasarlandı
Hedefe tekrar yönelim 8 sn	Lock state machine + zaman hedefi	Tasarlandı
5 dk takip testi	Dataset + Gazebo test senaryosu altyapısı	Tasarlandı
GPS manuel giriş	Python GUI uydu yönelim hesabı	Tasarlandı
Turksat 4B/5A yönelim	satellite_pointing.py geostationary hesap	Tasarlandı
Parametrelerin kalıcı saklanması	JSON config + ROS servisi	Tasarlandı
Bilgisayar/tablet arayuzu	Tkinter tabanlı Python GUI	Tasarlandı
CAD-Gazebo entegrasyonu	mergen_description mesh/URDF iskeleti	Tasarlandı

7. Gazebo Entegrasyon Yol Haritası

1. SolidWorks montajı tamamlanır, her parça ayrı mesh export edilir
2. Mesh dosyaları `mergen\_description/meshes/` altına yerleştirilir
3. URDF/Xacro: her parça link, hareketli bağlantılar joint
4. Azimuth: continuous joint, Elevasyon: revolute joint (0-90)
5. Stabilizasyon: 2x prismatic joint (CAD tamamlanana kadar)
6. IMU sensörü: Gazebo plugin ile terminal gövdesine bağlı
7. Motorlar: önce joint position controller, sonra ros2_control
8. Stewart platform: roll/pitch/yaw bozucu test node'u
9. Kontrol node'u: IMU + encoder + QPD -> motor komutları
10. Python arayüz: aynı topic/servislere bağlantı

8. KTR İçin Geliştirme Yol Haritası

1. Dataset sonuçlarıyla Kalman ve PID parametreleri ilk kez ayarlanır
2. CAD modeli mesh export edilir, URDF/Xacro modeline bağlanır
3. Gazebo içinde IMU, joint state ve motor kontrol pluginleri aktif
4. Stewart platform hareketi roll/pitch/yaw bozucu olarak simüle edilir
5. 5 dakika takip testi, 8 saniye yeniden kilitleme testi
6. Manuel/otomatik mod testleri kaydedilir
7. Gerçek donanım seçimi kesinlesince ROS node çıktıları Teensy/STM32 firmware arayüzüne bağlanır
8. KTR raporunda dataset, Gazebo ve donanım testleri aynı metriklerle sunulur

9. Çalıştırma Komutları

Windows:

```
# GUI başlatma<br>python arayuz/main.py<br><br># Dataset oluşturma ve doğrulama<br>python
Efe/tools/dataset_simulator.py --generate --output Efe/datasets/sample_motion.csv<br>python
Efe/tools/dataset_simulator.py --input Efe/datasets/sample_motion.csv --output
Efe/datasets/sample_result.csv<br><br># PDF rapor oluşturma<br>python Efe/reports/generate_pdf.py
```

Ubuntu 22.04 (ROS 2 Humble):

```
# ROS 2 workspace derleme<br>cd mergen_ws<br>colcon build<br>source install/setup.bash<br><br># GUI (ROS
bağlantılı)<br>python3 arayuz/main.py
```


10. Teknik Detaylar

PID Parametreleri

Eksen	Kp	Ki	Kd	Max Integral	Deadband	Wrap
Azimuth	2.5	0.1	0.3	20.0	0.1	Evet
Elevasyon	3.0	0.08	0.25	15.0	0.1	Hayir

QPD Hata Vektörü

$$ex = ((A + D) - (B + C)) / (A + B + C + D) \\ ey = ((A + B) - (C + D)) / (A + B + C + D)$$

Toplam sinyal < 0.05 ise hedef tespit edilememis kabul edilir.

ex < 0.03 ve ey < 0.03 ise hedef kilitlenmis kabul edilir.

Durum Makinesi Gecisleri

POWER_ON -> HW_SELF_TEST -> HOMING -> CALIBRATING -> IDLE
IDLE -> AUTO_TRACKING (otomatik mod)
IDLE -> MANUAL (manuel mod)
AUTO_TRACKING/MANUAL -> ERROR (hata durumu)
ERROR -> IDLE (hata cozuldu)

Hata Kodlari (bitmask)

Bit	Hata
0x0001	IMU_FAULT
0x0002	GPS_NO_FIX
0x0004	AZ_MOTOR_FAIL
0x0008	EL_MOTOR_FAIL
0x0010	TLE_INVALID
0x0020	AZ_LIMIT_HIT
0x0040	EL_LIMIT_HIT
0x0080	WATCHDOG

Sonuc

Mergen projesi; hareketli bir platform üzerinde uydu takibi yapabilmek için IMU tabanlı stabilizasyon, TLE/SGP4 ile yorunge hesabi, PID kontrol, lazer/QPD ile aktif geri besleme ve kullanıcı arayuzu gereksinimlerini karşılayacak şekilde tasarlanmıştır. Dört katmanlı mimari sayesinde algılama, kestirim, kontrol ve arayüz birbirinden bağımsız geliştirilebilmektedir. CAD modeli tamamlanana kadar dataset tabanlı doğrulama, CAD tamamlandıktan sonra Gazebo simülasyonu ve en son gömülü donanım entegrasyonu ile sistem doğrulama aşamaları tamamlanacaktır.